EC-341 Digital System Design

BS (CE)-2021

LAB REPORT #4

Submitted By:

Reg. No.	Name
21-CE-009	Huzaifa Shahbaz
21-CE-020	Muhammad Ibthesam
21-CE-021	Yawar Ali Malik

Department of Computer Engineering

HITEC University Taxila

	Presentation (Format(0.5)+ Title (0.5)+ Conclusion(1) +Procedure(2) +Objective(1))	Calculation/ Coding	Observation/ Program Results	Total
Total	**5**	**5**	**5**	**15**
Obtained				

_____

Lab Instructor.

Fasih Ahmad

Experiment #4

Implementation of Logic Gates in Xilinx

Objective:

1: Digital Circuits
2: Logic Gates
3: Lab Tasks

Equipment/Software Used:

Xilinx Vivado

Procedure:

Digital Circuits:

One caveat of FPGAs is that they can only create digital circuits. Some of the newer FPGAs include on-board analog to digital converters, but even these convert the analog input into a digital signal as soon as possible. But what is a digital circuit? In electronics, digital is used to describe circuits that abstract away continuous voltage values in favor of discrete 1s and 0s. The actual voltages used and the thresholds don't actually matter for the higher-level design but you will often see something like 0V being a 0 and 1.2V being a 1 inside the FPGA. If the actual voltage is, say, 0.8V that is close enough to 1.2V to be considered a 1 and everything works the same. A digital circuit is designed to push the voltages to the extremes which makes them incredibly resilient to noise and other real-world interference. The concept of digital also gives us a way to design complicated behavior into the circuit without having to worry about the lower-level design. We get to work in an ideal world. The nitty gritty is taken care of in the design of the simple building blocks that we will be using. These building blocks are logic gates.

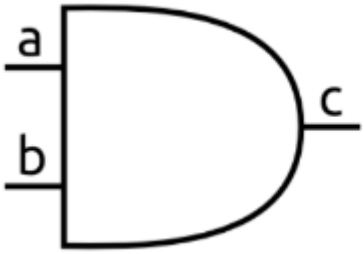
Logic Gates:

There are a handful of different logic gates but the most common ones are AND, OR, XOR, and NOT. Each of these takes digital inputs, performs its logical function, and outputs a digital value.

And gate:

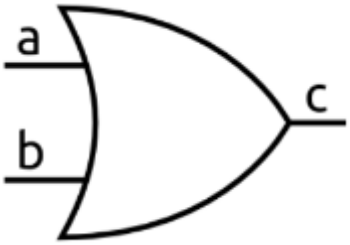
An AND gate takes two inputs and outputs a 1 only when the first input and the

second input are 1. If either input is 0, the output is 0. The symbol of an AND gate looks like this:



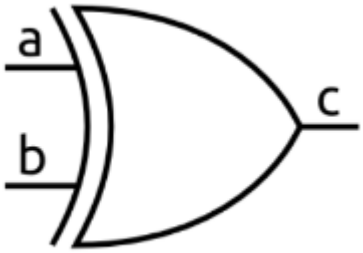
OR gate:

An OR gate takes two inputs and outputs a 1 when either the first input or the second input is 1. Only when both are 0 is the output 0. Here's the OR gate symbol:



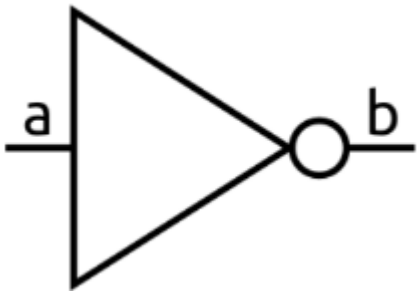
XOR gate:

An XOR gate is similar to an OR gate but only outputs a 1 when either the first input or the second input are 1, but not when both are 1. It can also be thought of as outputting a 1 when the inputs are different. The X in XOR stands for exclusive. Here is its symbol:

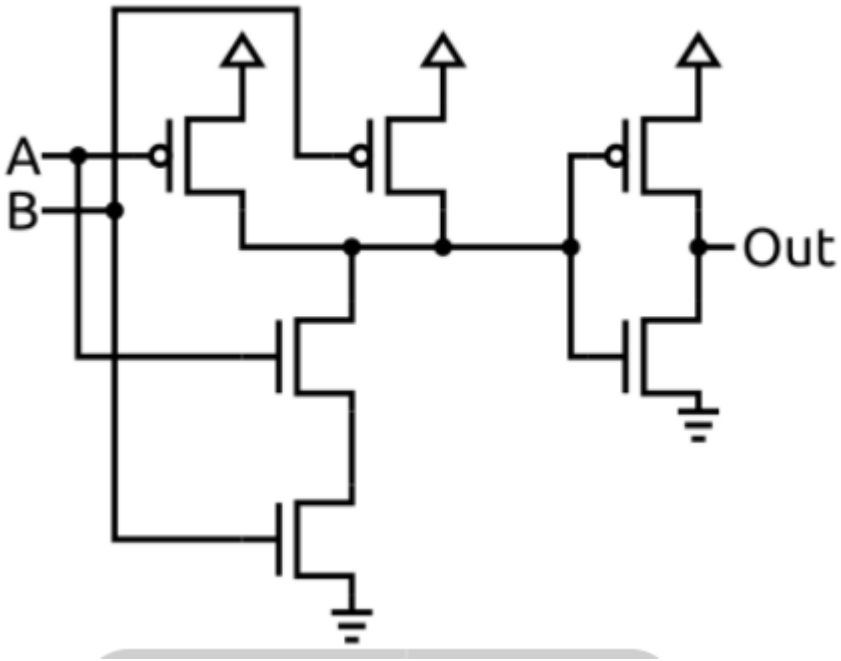


NOT Gate:

A NOT gate is the simplest gate. It has one input and simply outputs the opposite of whatever it is. So a 1 becomes a 0 and a 0 becomes a 1.



There are variations of the basic gates known as NAND, NOR, and XNOR. These are simply the standard versions with their outputs inverted. Just for some extra context, an AND gate, like all logic gates, can be built using transistors. The image below shows an example of how an AND gate could be implemented. The schematic uses NMOS and PMOS MOSFET transistors. This type of design is known as CMOS (complementary metal-oxide semiconductor) and is what is used in most modern circuits.



Tasks

Task #1:

Write a Gate Level Verilog Code for all basic logic gates. Run the code on ZYBO and verify results using different instances.

AND gate

BLOCK DIAGRAM:

BLOCK DIAGRAM OF AND GATE

Truth Table:

Truth Table OF AND GATE

A	B	C
0	0	0
0	1	0
1	0	0
1	1	1

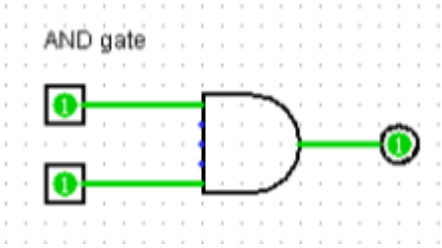
Boolean Expression:

A.B

Simplify Expression:

A.B

Logisim:

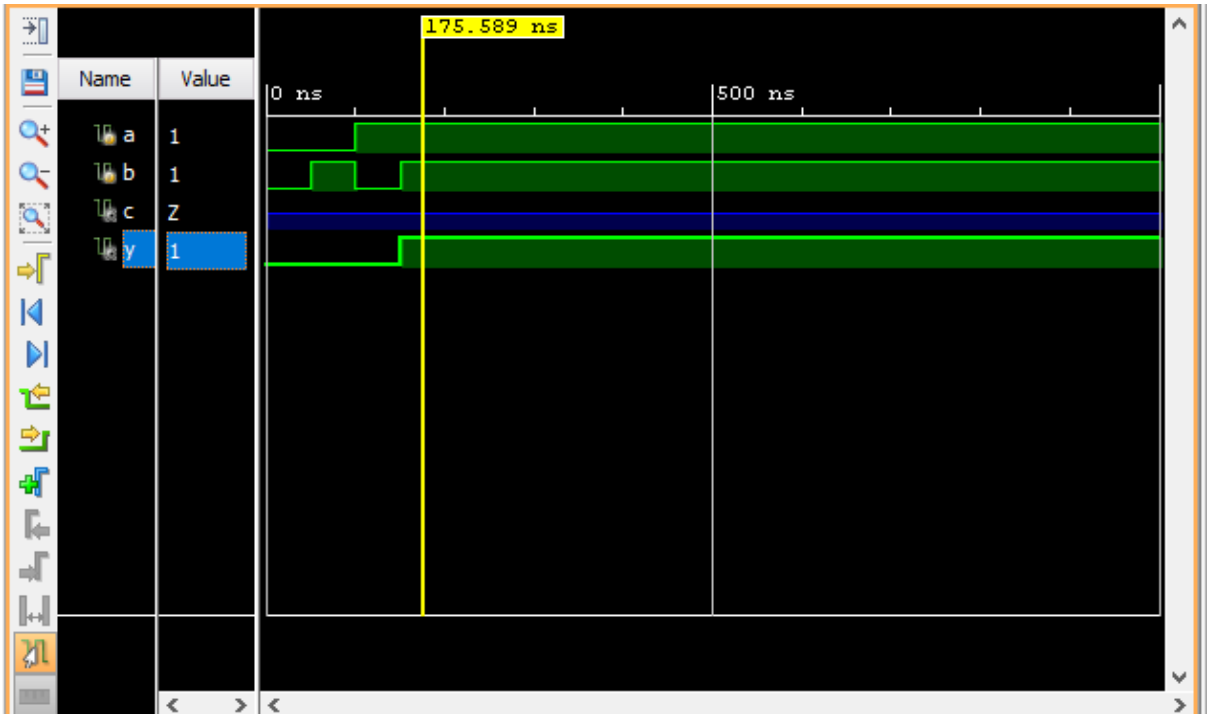


Verliog:

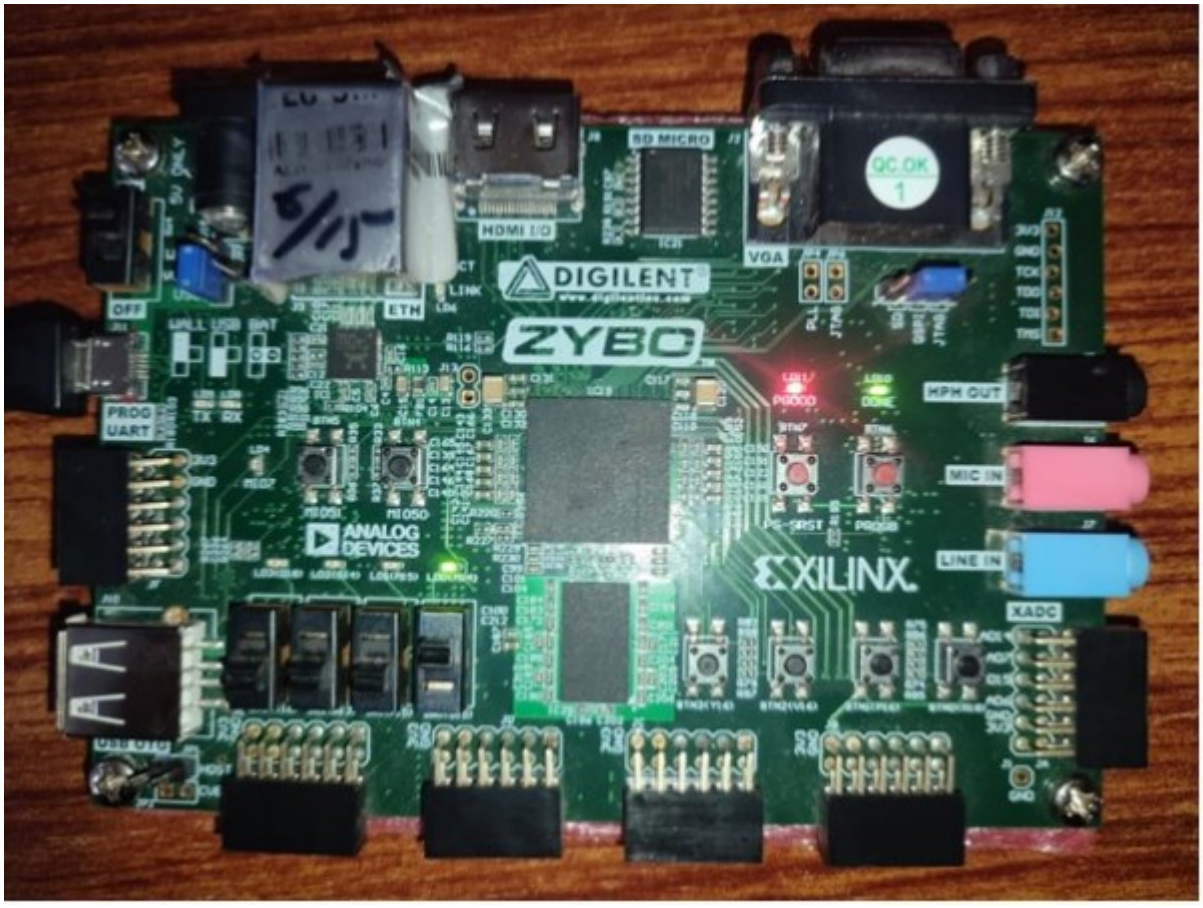
```
21 module andgate(
22     input a,
23     input b,
24     output c
25     );
26     assign c=a&b;
27 endmodule
```

```
21 module testbenchandgate();
22 reg a;
23 reg b;
24 wire c;
25 initial
26 begin
27 a=0;b=0;
28 #50 a=0;b=1;
29 #50 a=1;b=0;
30 #50 a=1;b=1;
31 end
32 endmodule
```

Waveform:



Zybo:



OR gate:

BLOCK DIAGRAM:

BLOCK DIAGRAM OF OR GATE

Truth Table:

Truth Table OF OR GATE

A	B	C
0	0	0
0	1	1
1	0	1
1	1	1

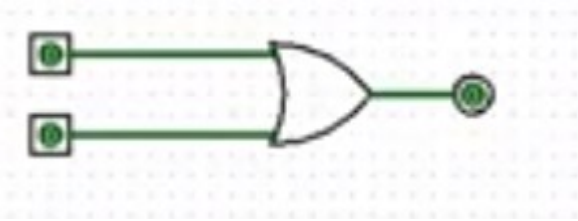
Boolean Expression:

A+B

Simplify Expression:

A+B

Logisim:



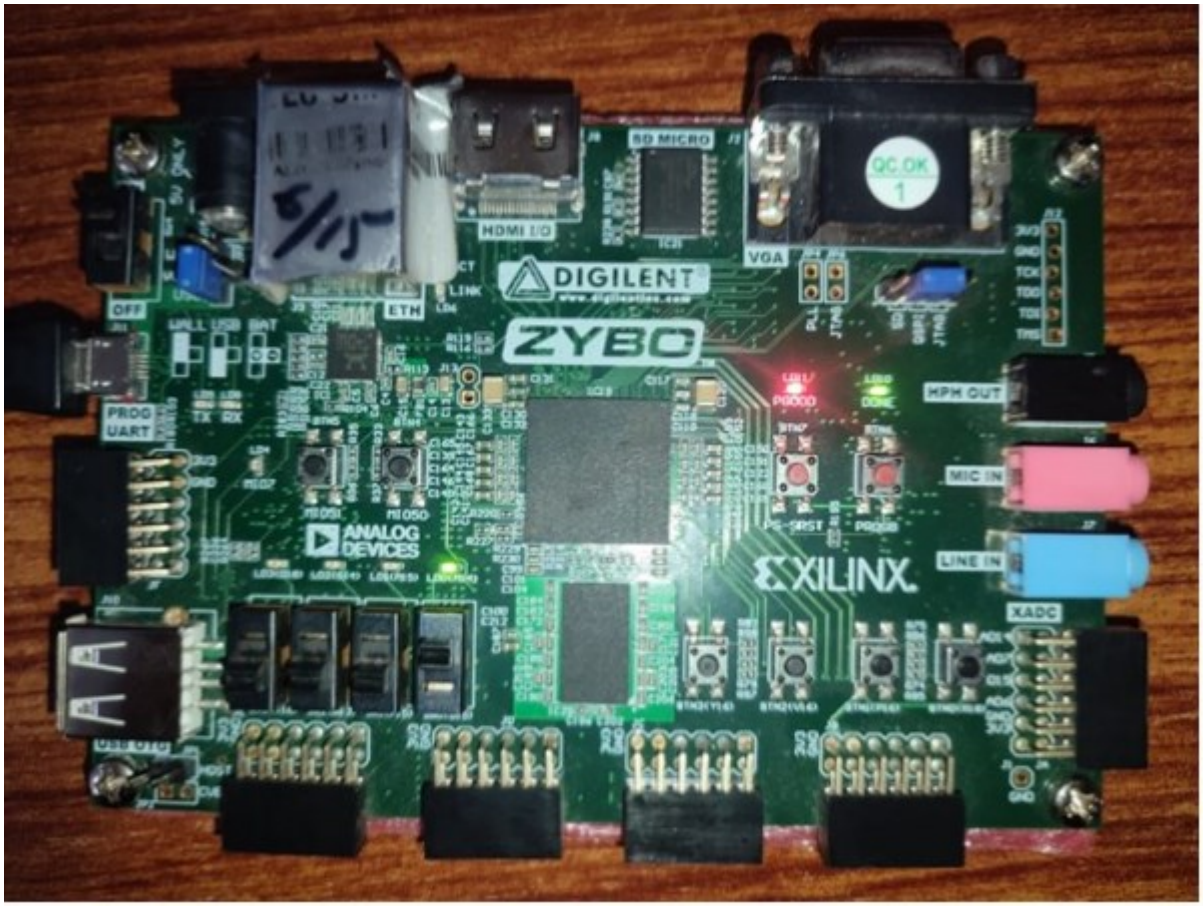
Verliog:

```
21 module orgate(
22     input a,
23     input b,
24     output c
25     );
26     always @ (a,b)
27     begin
28     y = a | b;
29 endmodule
```

```
21 module tborgate();
22   reg a, b;
23   wire c;
24   or_gate_s uut(a, b, c);
25   initial
26   begin
27   a=0;b=0;
28 #50 a=0;b=1;
29 #50 a=1;b=0;
30 #50 a=1;b=1;
31 end
32 endmodule
```

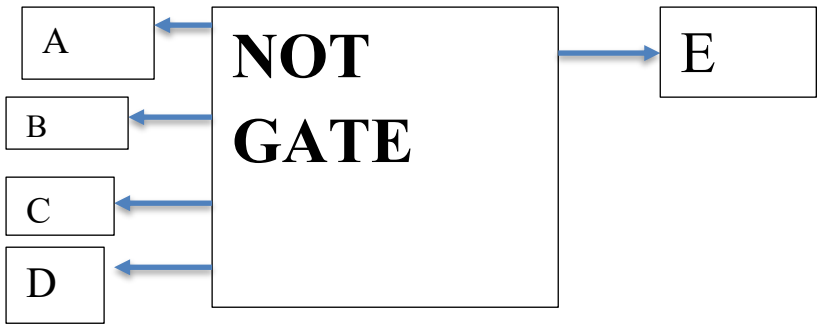
Waveform:

Zybo:



Not gate:

BLOCK DIAGRAM:



Block diagram of NOT GATE

Truth Table:

Truth table Of NOT GATE

Inputs				**Output**
A	**B**	**C**	**D**	**E**
0	0	0	0	0
0	0	0	1	1
0	0	1	0	1
0	0	1	1	1
0	1	0	0	1
0	1	0	1	1
0	1	1	0	1
0	1	1	1	1
1	0	0	0	1
1	0	0	1	1
1	0	1	0	1
1	0	1	1	1
1	1	0	0	1
1	1	0	1	1
1	1	1	0	1
1	1	1	1	0

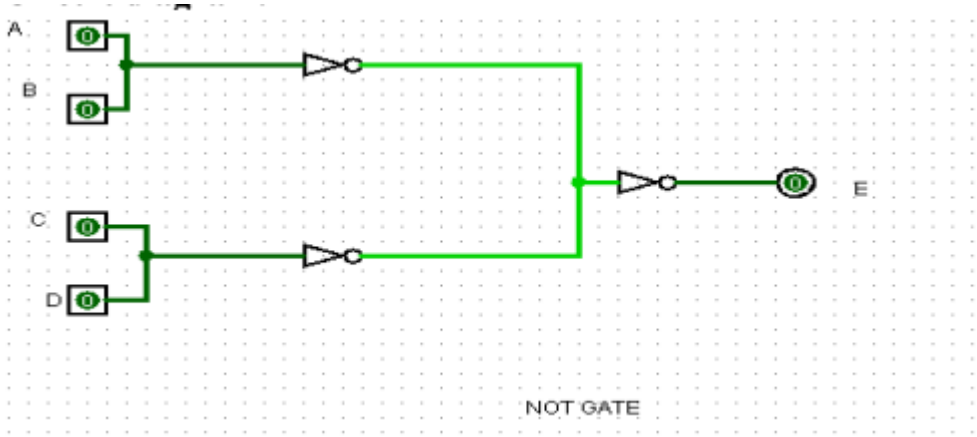
Boolean Expression:

A’+B’+C’+D’

Simplify Expression:

A’B’+C’D’

Logisim:

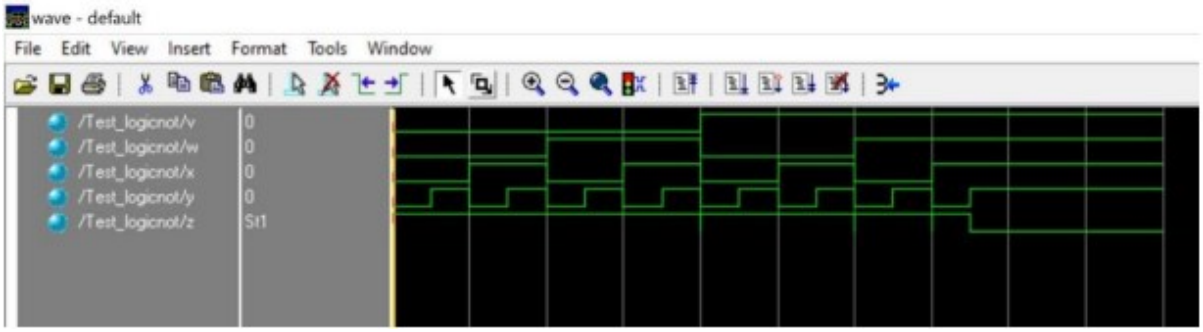


Verliog Code:

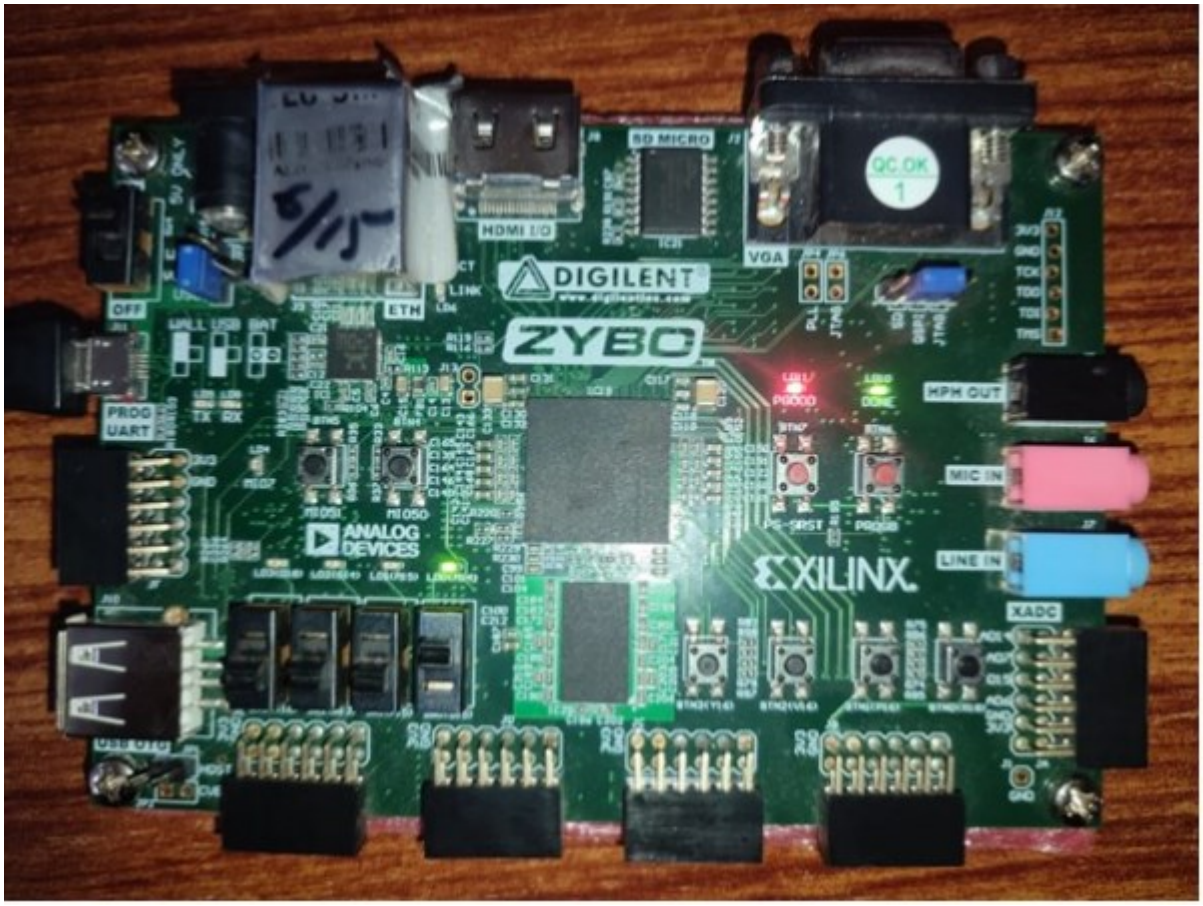
```
21 module notgate(
22     input a,
23     input b,
24     input c,
25     input d,
26     output e
27     );
28     not a1(e,a,b,c,d);
29 endmodule
```

```
21 module testnotgate();
22 reg a,b,c,d;
23 wire e;
24 nots notgate(a,b,c,d,e);
25 initial
26 begin
27 a=0;b=0;c=0;d=0
28 #50 a=0;b=0;c=0;d=1;
29 #50 a=0;b=0;c=1;d=0;
30 #50 a=0;b=0;c=1;d=1;
31 #50 a=0;b=1;c=0;d=0;
32 #50 a=0;b=1;c=0;d=1;
33 #50 a=0;b=1;c=1;d=1;
34 #50 a=1;b=0;c=0;d=0;
35 #50 a=1;b=0;c=0;d=1;
36 #50 a=1;b=0;c=1;d=1;
37 #50 a=1;b=1;c=1;d=1;
38 end
39 endmodule
```

Waveform:

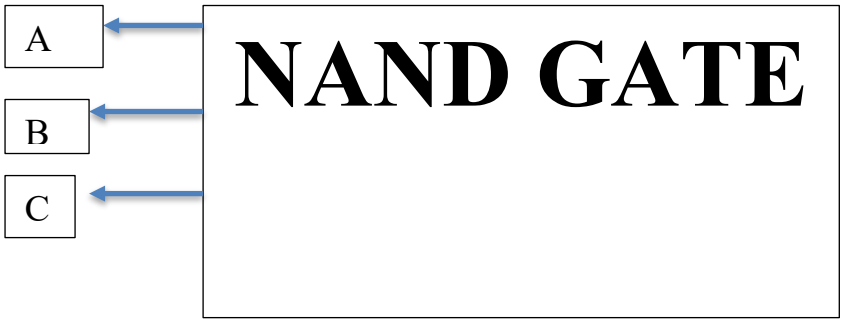


Zybo:



NAND GATE:

BLOCK DIAGRAM:



Task #2:

Conclusion (write 1 to 2 lines conclusion of experiment you performed in Times New Roman 12)